KumaWatch Patrol: An AI Robot That Patrols Bear-Sighting Risk

Risk-Aware Robotic Patrol for Bear Encounters

Author	**Todalaba**	Date	November 2026
Video URL	https://www.youtube.com/watch?v=t66S8h_LumE	Code	github.com/todalaba/KumaWatch-Patrol

1 Background and Objectives

Bears have been appearing in towns more and more often, and local governments are kept busy patrolling school routes, garbage collection points and similar places. Staff are limited, however, so they must choose only a few places to check tomorrow. The research project KumaWatch [7] aggregates the public sighting data of Yamagata and Akita Prefectures onto a 10 km grid and compares several forecasting methods on the task of selecting the top K cells with the highest sighting risk for the next day. A forecast, however, is only a number on a map; checking the site and acting safely if a bear is there is still left to people. This work makes that step, from forecast to on-site check, concrete with a robot.

1. Based on the AI's sighting-risk forecast, automatically choose as many sites as can be patrolled ($K = 3$) and the order in which to visit them.
2. Patrol the candidate sites autonomously with Navigation2, and find bears by checking camera images with AI.
3. On finding a bear, stop without chasing it, record its position, time and confidence, and then retreat to a safe spot away from the bear (the safe-side action).
4. Compare with a fixed-route patrol under the same conditions to show the effect of choosing sites with AI.

2 Proposed System

KumaWatch Patrol runs on a mobile robot (TurtleBot3 / Happy Mini) on ROS 2 Humble [4], Navigation2 [5] and Gazebo Classic 11 [6], and carries out the following steps automatically (Figure 1). (1) AI (TTM) forecasts the bear-sighting risk of 5 candidate sites. (2) Three sites and their order are chosen from risk, importance and travel distance. (3) The robot patrols them with Nav2. (4) AI (YOLOv8) detects bears in the camera image. (5) The robot stops, records the bear, retreats to a safe spot away from it, and continues. (6) It returns to the base and outputs the results to the terminal, RViz and files. AI is used in (1) and (4); the site selection in (2) is a rule (utility maximisation) that takes the AI forecast as input. The town and sites are fictional, and bears, deer and buildings are built from primitive shapes. The risks are TTM forecasts based on public data; the training images for bear detection are synthetic images made in the simulator.

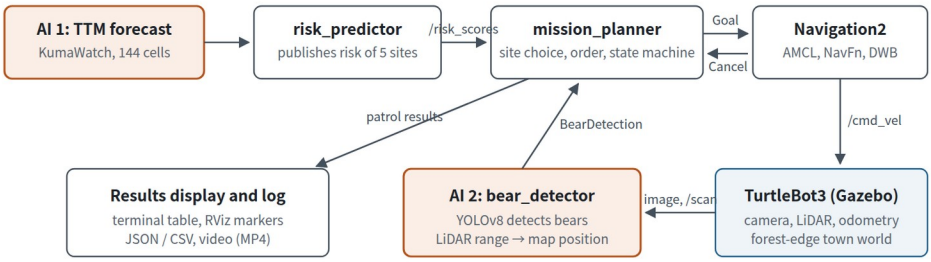


Figure 1 System architecture. Orange boxes are the parts that use AI. Nodes are connected by ROS 2 topics (custom messages for risk and bear detection) and the Nav2 NavigateToPose action.

Sites are chosen by a utility that considers not only risk but also social importance and travel cost.

$$U_i = w_i\, p_i - \lambda\, d_i$$

p_i is the risk, w_i the importance (school route 1.2, residential 1.1, others 1.0), d_i the distance from the base [m], and $\lambda = 0.02$. The top 3 sites by U_i are selected, and the order that minimises the total distance of the round trip from the base is found over all permutations. In this case the route is P2 (forest edge) → P4 (garbage point) → P3 (school route). For comparison, risk-only selection and a fixed route are also implemented.

`mission_planner` moves to each site with the Nav2 NavigateToPose action and observes the surroundings on arrival. When it receives a bear detection it cancels the goal, stops with zero velocity, and records the bear's position, time, distance and confidence. It does not chase the bear: it computes a safe spot 3 m away in the direction opposite the bear, retreats there with Nav2, and resumes the patrol. If no path to the safe spot is found, this prototype abandons the retreat and continues the patrol. This is a safety-control issue (Sections 5, 6 and 8).

3 How AI Is Used

3.1 AI 1: Forecasting bear-sighting risk (KumaWatch's TTM)

For risk we used the forecasts of **IBM Granite TinyTimeMixer (TTM 1536-96-R2) [1]** precomputed in KumaWatch (only the values for the 5 sites are included). TTM is a foundation model for time-series forecasting pre-trained on a large number of time series; KumaWatch uses it to forecast, zero-shot, the sightings on every day of 2025 (not used for training) from the history of daily sighting counts in 144 cells of Yamagata Prefecture. The town in this work is fictional, but the risks of the 5 sites are forecasts for real cells (Table 2). They were assigned by the following rule: the 2 cells with an actual sighting on 7 June 2025 become P2 and P4, and bears are placed there in Gazebo as well. From the cells without a sighting we chose the one ranked highest by TTM (P3, an AI false alarm), one around 40th (P5) and one around 100th (P1). Risk is the cell's rank among the 144 cells of the prefecture, expressed as a percentile.

Site	Setting	Cell	TTM rank	Risk	Actual sighting	Bear in Gazebo
P4	Garbage point	5_1	1 / 144	1.000	Yes	Yes
P3	School route	4_1	2 / 144	0.993	No (false alarm)	No
P2	Forest edge	1_10	4 / 144	0.979	Yes	Yes
P5	Riverside	6_0	40 / 144	0.727	No	No (deer)
P1	Residential	1_0	93 / 144	0.357	No	No

Table 2 TTM forecasts assigned to the 5 sites (7 June 2025, Yamagata Prefecture 10 km grid).

3.2 AI 2: Detecting bears in camera images (YOLOv8)

COCO-pretrained YOLOv8 [2] already has a "bear" class, but it mistook the Gazebo bears (built from primitive shapes) for traffic lights and people (Figure 2, left). We therefore **fine-tuned** it on Gazebo images. We took 700 images while moving the camera, bears and deer to random positions, and labelled the correct boxes automatically by projecting each bear's 3D position into the image. Viewpoints where the bear is hidden by trees or buildings were excluded, and 30% of the images contain no bear. Fine-tuning started from the pretrained YOLOv8n (Figure 2, right).

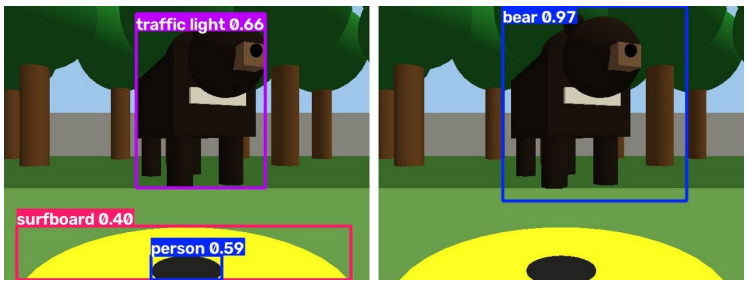


Figure 2 Before and after fine-tuning. Left: COCO-pretrained YOLOv8s mistakes the Gazebo bear for a "traffic light". Right: the fine-tuned YOLOv8n labels it "bear" (confidence 0.97).

`bear_detector` computes the bear's bearing relative to the camera from the horizontal position of the box, and its distance from the LiDAR reading in that direction; combined with the robot's own pose, this gives the bear's position on the map. If the LiDAR hits a tree in front, it switches to an estimate from the box height. To avoid stopping on false detections, it asks the robot to stop only after 3 consecutive detections within 4 m.

Precision (105 validation images)	Recall (105 validation images)	mAP@0.5	Patrol viewpoints
0.94	0.99	0.985	11 / 11

Table 3 Accuracy of YOLOv8 bear detection (105 validation images from Gazebo, and 11 patrol viewpoints).

On 11 patrol viewpoints (approaching and arriving at P2 and P4; bear-free P1, P3 and P5; the deer; the forest edge, etc.), every viewpoint showing a bear was detected (confidence 0.88–0.98), and deer, trees and buildings were never mistaken for bears. As a ROS 2 node, we also confirmed that it detects bears in the Gazebo camera stream and publishes the stop message.

4 ROS 2 Node Architecture

The software consists of five ROS 2 packages (Table 1). The world and the Nav2 map are both generated automatically from the same definition file (`world_layout.yaml`), so their coordinates match without building a map by SLAM. Bears and deer are not on the map; the robot finds them with the LiDAR and camera as unknown obstacles.

Node	Role
risk_predictor	Publishes the TTM forecasts as the risk of the 5 sites and shows them as bars in RViz
mission_planner	Chooses the sites and visiting order, sends goals to Nav2, stops, records and retreats to a safe spot on bear detection, shows results (state machine)
bear_detector	Checks camera images with YOLOv8 and estimates the bear's map position from the LiDAR range
image_recorder	Records the camera and the detection-box video with a status overlay; saves a still when a bear is detected
bear_proximity_monitor	Simple detector using the distance to known bear coordinates (for comparison and testing)

Table 1 Main nodes. The state machine moves through RISK_ASSESSMENT → ROUTE_PLANNING → PATROL ⇄ OBSERVE → (EMERGENCY_STOP) → RETURN_TO_BASE → FINISHED.

5 Results in Gazebo

In the lecture environment (Docker image airobotbook/ros2-desktop-ai-robot-book-humble, Happy Mini, no GPU) we ran the AI route once with Gazebo, Navigation2 and YOLOv8 all running (Figure 3 and the submission video). The robot left the base for P2, and YOLOv8 detected a bear in the camera image (confidence 0.75, 4.0 m away). The robot stopped, recorded the position, time and confidence, and tried to retreat to a safe spot. On the way to P4 it detected a second bear (confidence 0.82, 3.9 m), stopped, recorded it, and retreated to a safe spot. It then reached P3, confirmed there was no bear, and returned to the base. All 3 sites were checked; 2 bears were found, the distance travelled was 31.4 m, and the time taken was 173 s (simulation time). Part of the robot body (a yellow band and the back of a logo) appears in Happy Mini's camera image, but the bears were still detected.



Figure 3 The run in Gazebo (from the submission video). Left: the robot's camera image with the YOLOv8 box (the bear at P2, confidence 0.87 at 5.0 m, before the 4 m stopping distance); right: a map drawn from the logged robot pose. The top bar shows the patrol state (Mission 1/3, AI risk).

At P2, path planning to the safe spot failed because of the obstacle layout. The retreat target (−2.7, −2.2) was next to a building, and Nav2 could not make a path. This prototype then switched to continuing the patrol, but in real operation a safe stop, a search for an alternative retreat spot, and a notification to the operator must take priority. This is a safety-control issue to be fixed.

6 Comparison with a Fixed Route

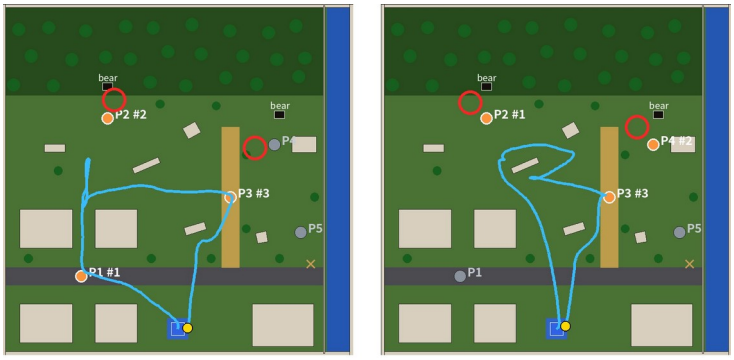


Figure 4 Trajectory comparison. Drawn from the logs of real runs in the lecture Docker environment (Gazebo, Navigation2, YOLOv8). Left: fixed patrol (P1→P2→P3); right: AI patrol (P2→P4→P3). Blue lines are the trajectories, red circles are where bears were found, and the distances are the logged values.

Metric	Fixed patrol (P1→P2→P3)	AI patrol (P2→P4→P3)
Summed AI risk of the patrolled sites	2.33	2.97
Sites checked	3 / 3	3 / 3
Bears found	2 (1 near P4, seen by chance from P3)	2 (while checking P2 and P4)
YOLOv8 confidence at detection	0.73, 0.86	0.75, 0.82
Retreat to a safe spot	Succeeded both times	P4 succeeded; P2 failed (no path)
Distance travelled (log)	33.6 m	31.4 m
Time taken (simulation time)	166.6 s	173.3 s

Table 4 Comparison with the same 5 candidate sites and the same bear placement (one run each, including the return to base). Values from real runs in the lecture Docker environment with Gazebo, Navigation2 and camera-based bear detection by YOLOv8. Distance and time are the logged values (patrol_summary.json); the comparison screen in the video uses the same values.

- **The comparison is one simulation run per condition.** It does not prove a general advantage; it is a demonstration of the whole Physical AI pipeline (forecast → site selection → autonomous patrol → detection → stop and retreat → return). Future work will evaluate multiple runs with different bear placements and initial conditions.
- **The fixed route's second bear was the bear near P4, detected by chance from P3.** The fixed route does not include P4; the AI route checked P4 as an explicit patrol target.
- **When path planning to the safe spot fails, real operation must give priority to a safe stop and a notification to the operator.** This prototype switched to continuing the patrol at P2; this is a safety-control issue to be fixed (Sections 5 and 8).

Both patrols found 2 bears, but the results mean different things. The AI patrol chose P4 (garbage point), the site with the highest risk, and detected the bears while checking P2 and P4. The fixed patrol's second bear was the bear near P4, which happened to come into view when the robot reached P3 (school route) (judged 3.2 m away, estimated position (4.4, 1.8)); this does not mean that the AI patrol failed. The summed risk of the sites chosen by the AI patrol is 2.97, higher than the fixed patrol's 2.33, which includes the low-risk P1. Distance and time were similar.

The fixed patrol was run twice: in the first run, path planning got stuck after the retreat at P2, so the robot could not reach P3 or return (1 bear). The table shows the second run, which completed. In the year-long evaluation, TTM is significantly worse in Yamagata than a simple static forecast (the historical average). Distances are the logged values (accumulated by mission_planner); the video's comparison screen uses the same values.

7 Design Points

- **Use the research forecasts as they are.** We used KumaWatch's actual TTM forecasts and placed bears in the cells with actual sightings, including an AI false alarm (P3) on purpose.
- **Consider importance and travel distance, not only danger.** Sites are chosen by a utility with a school-route weight and travel cost, and visited in the order with the shortest total distance.

- **Fine-tuning with automatic labelling.** Correct boxes are made by projecting each bear's 3D position into the image, giving 700 training images without manual work.
- **Safe-side behaviour.** The robot does not approach a bear; it stops within 4 m, records it and retreats to a safe spot. Requiring 3 consecutive detections prevents stops on false detections.
- **World and map generated from one definition.** The map always matches the layout. Localisation drift was fixed by adding poles and benches as landmarks.

8 Limitations and Future Work

- **Scope of the simulation.** This is a simulated environment in Gazebo; localisation, communication and weather in real environments have not been tested.
- **Bear detection was trained and evaluated only on Gazebo bears.** Accuracy on real images is unknown; real-world use needs training and evaluation on real images.
- **The mapping between forecasts and sites is illustrative.** TTM forecasts are per 10 km cell, not a single spot in a town, and the date is a single day chosen for the scenario.
- **Safety control when retreat is impossible.** Returning to normal patrol near a bear is a weak safety design. It should be changed to a safe stop, a search for another safe spot, a notification to the operator, or waiting for a human decision. After the run, alternative safe spots and a safe stop (SAFE_HOLD: stop, alert and wait for the operator) were implemented in the repository (verified with the minimal simulator; not yet re-run in Gazebo).
- **Next steps.** Re-run in Gazebo with SAFE_HOLD; fix the robot body appearing in the camera; compare patrol results with GLM-Logit and the static forecast; evaluate multiple runs with different bear placements and initial conditions.

9 Software and References

Environment: Ubuntu 22.04, ROS 2 Humble, Gazebo Classic 11, Navigation2, TurtleBot3 / Happy Mini, Ultralytics YOLOv8; runs in the lecture Docker environment. See README.md and training/ in the repository for launch steps, data and training programs.

External software and assets: KumaWatch [7] (only precomputed TTM scores for the 5 sites are included; source data CC BY 4.0, Yamagata Prefecture's open wildlife sighting data; no municipal data or non-public research code), IBM Granite TinyTimeMixer (Apache-2.0), Ultralytics YOLOv8 (AGPL-3.0; the fine-tuned weights are under the same licence), ROS 2, Navigation2 and Gazebo (Apache-2.0), TurtleBot3 simulation (ROBOTIS, Apache-2.0), Happy Mini (Introduction to AI Robots). Bear, deer, building and tree models were built from primitive shapes (no external 3D models). Generative AI (Anthropic's Claude) was used to help write the code and documents; the task design, the runs in the lecture environment and the checking of the results were done by the author.

References

[1] V. Ekambaram, A. Jati, P. Dayama, et al., "Tiny Time Mixers (TTMs): Fast Pre-trained Models for Enhanced Zero/Few-Shot Forecasting of Multivariate Time Series," NeurIPS, 2024.

[2] G. Jocher, A. Chaurasia, J. Qiu, "Ultralytics YOLOv8," 2023. https://github.com/ultralytics/ultralytics

[3] T.-Y. Lin, M. Maire, S. Belongie, et al., "Microsoft COCO: Common Objects in Context," ECCV, 2014.

[4] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," Science Robotics, 7(66), 2022.

[5] S. Macenski, F. Martín, R. White, J. Ginés Clavero, "The Marathon 2: A Navigation System," IROS, 2020.

[6] N. Koenig, A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," IROS, 2004.

[7] KumaWatch (research on bear-sighting risk forecasting; private research repository).